Passwordless SSH Authentication Between Two Linux/EC2 Servers

Problem: Servers often need to talk to each other for tasks like backups, rsync, CI/CD deployments, and config management (Ansible, etc.). Password-based SSH breaks automation since someone has to manually type the password every time — this fails for cron jobs/scripts, creates security risks (guessing, leaks, brute-force), and doesn't scale well across many servers.

Solution: SSH Public-Key Authentication, using a cryptographic key pair instead of passwords:

- **Private key** (`id_rsa`) — stays on the source server, never shared, proves identity.
- **Public key** (`id_rsa.pub`) — copied to the destination server's `~/.ssh/authorized_keys`, used to verify identity.

How it works:

1. Server A generates a key pair.
2. Server A's public key is added to Server B's `authorized_keys` file.
3. When Server A connects, Server B checks if a matching public key exists, issues a cryptographic challenge, and grants access if Server A proves it holds the matching private key — no password needed.

Production use cases: automated backups (rsync), CI/CD pipelines (Jenkins, GitLab CI, GitHub Actions), config management tools (Ansible, Chef, Puppet), and unattended file transfers (scp/sftp/rsync).

Lab setup: Server A (172.31.31.174) initiates the connection; Server B (172.31.22.11) accepts it. Port 22 must be open in the security group.

Implementation steps:

1. SSH into Server A, run `ssh-keygen -t rsa` to generate the key pair.
2. Verify keys exist in `~/.ssh` (`id_rsa`, `id_rsa.pub`).
3. Copy the public key with `cat ~/.ssh/id_rsa.pub`.
4. SSH into Server B, create `~/.ssh` if missing, and paste the public key into `~/.ssh/authorized_keys`.
5. If authentication fails due to permission errors, fix them:
    - `chmod 700 ~/.ssh` (owner: read/write/execute only)
    - `chmod 600 ~/.ssh/authorized_keys` (owner: read/write only)
6. From Server A, SSH into Server B again — it should log in without a password prompt.

7. Validate with `ssh -v` and look for "Offering public key" / "Authentication succeeded" in the verbose output.

Benefits over password auth: better security, full automation support, easier scaling, and a seamless user experience — making it the standard for backups, deployments, and CI/CD in production.

Conclusion: By keeping the private key on the source and the public key on the destination, passwordless SSH removes manual password entry, strengthens security, and enables reliable automation — which is why it's widely used in production server-to-server workflows.